

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

**Duration :60 Mins
Off Class
Total Marks:50**

Annexure A

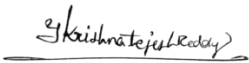
DESIGN TASK

Code of Conduct:

I_Y . Krishna Tejesh Reddy(192411200)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTION 1

Design a parser to recognize simple for loops with compound statements.

Grammar

Loop $\rightarrow$ for(A;C;I) Block

Block $\rightarrow$ { StmtList }

StmtList $\rightarrow$ Stmt

StmtList $\rightarrow$ Stmt StmtList

Stmt $\rightarrow$ id = E ;

A $\rightarrow$ id = num

C $\rightarrow$ id < num

I $\rightarrow$ id++

E $\rightarrow$ id

E $\rightarrow$ num

Input

```
for(i=0;i<5;i++)
{
x=10;
y=20;
}
```

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the loop statement.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Program

File 1: ForLoop.g4

```
grammar ForLoop;
```

```
start : loop EOF;
```

```
loop
```

```
    : FOR '(' init ';' cond ';' inc ')' block
```

```
    ;
```

block

```
    : '{' stmtList '}'
```

```
    ;
```

stmtList

```
    : stmt
```

```
    | stmt stmtList
```

```
    ;
```

stmt

```
    : ID '=' expr ';' 
```

```
    ;
```

init

```
    : ID '=' NUM
```

```
    ;
```

cond

```
    : ID '<' NUM
```

```
    ;
```

inc

```
    : ID '++'
```

```
    ;
```

expr

```
    : ID
```

```
    | NUM
```

```
    ;
```

FOR : 'for';

ID : [a-zA-Z][a-zA-Z0-9]*;

NUM : [0-9]+;

WS : [\t\r\n]+ -> skip;

File 2: Main.java

```
import org.antlr.v4.runtime.*;
```

```

public class Main {
    public static void main(String[] args) throws Exception {

        CharStream input = CharStreams.fromStream(System.in);

        ForLoopLexer lexer = new ForLoopLexer(input);
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        ForLoopParser parser = new ForLoopParser(tokens);

        parser.removeErrorListeners();

        parser.addErrorListener(new BaseErrorListener() {
            @Override
            public void syntaxError(
                Recognizer<?, ?> recognizer,
                Object offendingSymbol,
                int line,
                int charPositionInLine,
                String msg,
                RecognitionException e) {

                System.out.println("Rejected");
                System.exit(0);
            }
        });

        parser.start();

        System.out.println("Accepted");
    }
}

```

Procedure to Run

Open the VS Code terminal in:

E:\Compiler design

Step 1: Generate Parser

```
java -jar ".\antlr-4.13.2-complete.jar" ForLoop.g4
```

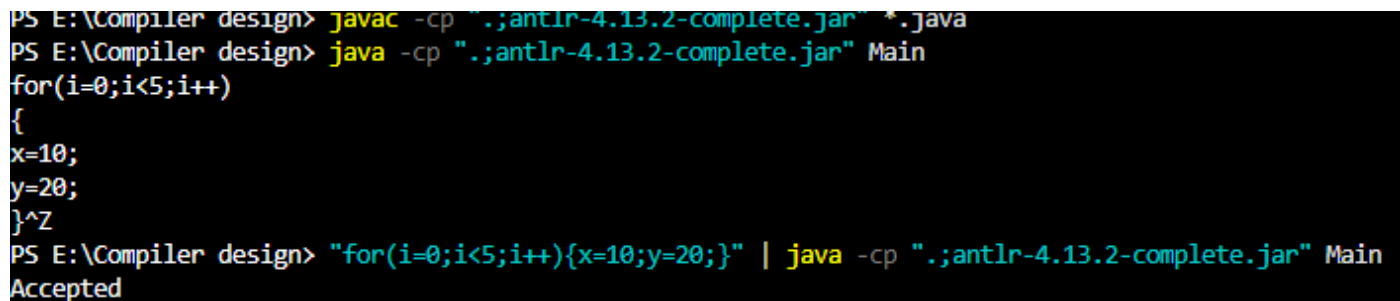
Step 2: Compile

```
javac -cp ".;antlr-4.13.2-complete.jar" *.java
```

Step 3: Run

```
"for(i=0;i<5;i++){x=10;y=20;}" | java -cp ".;antlr-4.13.2-complete.jar" Main
```

Output



```
PS E:\Compiler design> javac -cp ".;antlr-4.13.2-complete.jar" *.java
PS E:\Compiler design> java -cp ".;antlr-4.13.2-complete.jar" Main
for(i=0;i<5;i++)
{
x=10;
y=20;
}^Z
PS E:\Compiler design> "for(i=0;i<5;i++){x=10;y=20;}" | java -cp ".;antlr-4.13.2-complete.jar" Main
Accepted
```

Result

The given for loop with compound statements is successfully recognized by the parser and the output is:

Accepted

QUESTION 2

Title

Design a parser to recognize simple recursive grammar for balanced strings.

Grammar

$S \rightarrow aSb$

$S \rightarrow ab$

Input

aaabbb

Files Created

Inside:

E:\Compiler design\Quetion 2

we created:

antlr-4.13.2-complete.jar

BalancedString.g4

Main1.java

ANTLR automatically generated:

BalancedStringLexer.java

BalancedStringParser.java

BalancedStringListener.java

BalancedStringBaseListener.java

BalancedString.g4

The grammar recognizes strings where every a is matched with a corresponding b.

grammar BalancedString;

start

: S EOF

;

S

: 'a' S 'b'

| 'a' 'b'

;

WS

: [\t\r\n]+ -> skip

;

Main1.java

The Java program:

- Reads the input.
- Performs lexical analysis using the ANTLR lexer.
- Sends tokens to the parser.
- Detects syntax errors.
- Displays **Accepted** for valid strings.
- Displays **Rejected** for invalid strings.

Commands Used

Generate parser

```
java -jar ".\antlr-4.13.2-complete.jar" BalancedString.g4
```

Compile

```
javac -cp ".;antlr-4.13.2-complete.jar" *.java
```

Run

```
"aaabbb" | java -cp ".;antlr-4.13.2-complete.jar" Main1
```

Output

```
PS E:\Compiler design\Question 2> javac -cp ".;antlr-4.13.2-complete.jar" *.java
PS E:\Compiler design\Question 2> "aaabbb" | java -cp ".;antlr-4.13.2-complete.jar" Main1
Accepted
PS E:\Compiler design\Question 2> "aaabb" | java -cp ".;antlr-4.13.2-complete.jar" Main1
line 1:0 token recognition error at: 'aaabb\r'
Rejected
```

QUESTION 3

Design a parser to recognize nested arithmetic expressions with exponent operator.

Grammar

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow F ^ G$

$F \rightarrow G$

$G \rightarrow \text{id}$

$G \rightarrow \text{num}$

$G \rightarrow (E)$

Input

$(a+b)^2*c$

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Apply operator precedence.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Files Created

Inside:

E:\Compiler design\Question 3

we created:

antlr-4.13.2-complete.jar

Expression.g4

Main2.java

ANTLR generated:

ExpressionLexer.java

ExpressionParser.java

ExpressionListener.java

ExpressionBaseListener.java

Parser Grammar Used

The original grammar contains left recursion, so it was rewritten in ANTLR-compatible form:

```
grammar Expression;
```

```
start
```

```
  : E EOF
```

```
  ;
```

```
E
```

```
  : T ('+' T)*
```

```
  ;
```

```
T
```

```
  : F ('*' F)*
```

```
  ;
```

```
F
```

```
  : G ('^' G)*
```

```
  ;
```

```
G
```

```
  : ID
```

```
  | NUM
```

```
  | '(' E ')'
```

```
  ;
```

```
ID
```

```
  : [a-zA-Z][a-zA-Z0-9]*
```

```
  ;
```

```
NUM
```

```
  : [0-9]+
```

```
  ;
```

```
WS
```

```
  : [ \t\r\n]+ -> skip
```

```
  ;
```

This maintains the precedence:

+ → lowest
* → higher
^ → higher
() → highest

Execution Commands

Generate parser

```
java -jar ".\antlr-4.13.2-complete.jar" Expression.g4
```

Compile

```
javac -cp ".;antlr-4.13.2-complete.jar" *.java
```

Execute

```
"(a+b)^2*c" | java -cp ".;antlr-4.13.2-complete.jar" Main2
```

Output

```
Accepted
PS E:\Compiler design\Question 3> javac -cp ".;antlr-4.13.2-complete.jar" *.java
PS E:\Compiler design\Question 3> "(a+b)^2*c" | java -cp ".;antlr-4.13.2-complete.jar" Main2
Accepted
PS E:\Compiler design\Question 3> "(a+b)^2*" | java -cp ".;antlr-4.13.2-complete.jar" Main2
Rejected
```

Result

Thus, the parser successfully recognizes nested arithmetic expressions with the required operator precedence and displays Accepted or Rejected for valid and invalid expressions.

QUESTION 4

Design a parser to recognize SQL SELECT statements with WHERE clause.

Grammar

Query $\rightarrow$ SELECT Columns FROM id WHERE Condition ;

Columns $\rightarrow$ *

Columns $\rightarrow$ id

Columns $\rightarrow$ id , Columns

Condition $\rightarrow$ id Relop value

Relop $\rightarrow$ = | < | > | <= | >=

Input

SELECT name,age FROM Student WHERE age>=18;

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the SQL query.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Files Created

Inside:

E:\Compiler design\Question 4

we created:

antlr-4.13.2-complete.jar

SQLQuery.g4

Main3.java

ANTLR generated:

SQLQueryLexer.java

SQLQueryParser.java

SQLQueryListener.java

SQLQueryBaseListener.java

Parser Function

The parser checks:

- SELECT keyword
- Column names
- FROM keyword
- Table name
- WHERE keyword
- Condition
- Relational operator such as =, <, >, <=, >=
- Value
- Semicolon ;

The input must follow the specified SQL structure.

Execution Commands

Generate parser

```
java -jar ".\antlr-4.13.2-complete.jar" SQLQuery.g4
```

Compile

```
javac -cp ".;antlr-4.13.2-complete.jar" *.java
```

Execute

```
"SELECT name,age FROM Student WHERE age>=18;" | java -cp ".;antlr-4.13.2-complete.jar" Main3
```

Output

```
PS E:\Compiler design\Qution 4> java -jar ".\antlr-4.13.2-complete.jar" SQLQuery.g4
PS E:\Compiler design\Qution 4> javac -cp ".;antlr-4.13.2-complete.jar" *.java
PS E:\Compiler design\Qution 4> "SELECT name,age FROM Student WHERE age>=18;" | java -cp ".;antlr-4.13.2-complete.jar" Main3
Accepted
PS E:\Compiler design\Qution 4> "SELECT name,age FROM Student WHERE age>;" | java -cp ".;antlr-4.13.2-complete.jar" Main3
Rejected
```

Result

Thus, the parser successfully recognizes SQL SELECT statements with a WHERE clause and displays Accepted for valid queries and Rejected for invalid queries.

QUESTION 5

Design a parser to recognize a complete mini programming language with declarations, assignment, conditional statements, loops, function calls, and return statement.

Grammar

$\text{Program} \rightarrow \text{int main()}\{\text{DeclList StmtList Return}\}$

$\text{DeclList} \rightarrow \text{Decl DeclList} \mid \varepsilon$

$\text{Decl} \rightarrow \text{int id} ;$

$\text{StmtList} \rightarrow \text{Stmt StmtList} \mid \varepsilon$

$\text{Stmt} \rightarrow \text{id} = \text{num} ;$

$\text{Stmt} \rightarrow \text{id} = \text{id} ;$

$\text{Stmt} \rightarrow \text{if}(\text{E}) \text{ Stmt else Stmt}$

$\text{Stmt} \rightarrow \text{while}(\text{E}) \text{ Stmt}$

$\text{Stmt} \rightarrow \text{id}(\text{id});$

$\text{Return} \rightarrow \text{return num} ;$

$\text{E} \rightarrow \text{id} < \text{num}$

$\text{E} \rightarrow \text{id} > \text{num}$

Input

```
int main()
{
int x;
x=10;
while(x>0)
display(x);
if(x>5)
x=1;
else
x=0;
return 0;
}
```

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the complete program.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Files Created

Inside:

E:\Compiler design\Qution 5

we created:

antlr-4.13.2-complete.jar

MiniLang.g4

Main4.java

ANTLR generated:

MiniLangLexer.java

MiniLangParser.java

MiniLangListener.java

MiniLangBaseListener.java

Parser Function

The parser validates:

- int main() structure
- Variable declarations
- Assignment statements
- while loops
- if-else statements
- Function calls
- Relational expressions
- return statement
- Required semicolons and brackets
- Syntax errors

Execution Commands

Generate parser

```
java -jar ".\antlr-4.13.2-complete.jar" MiniLang.g4
```

Compile

```
javac -cp ".;antlr-4.13.2-complete.jar" *.java
```

Execute

```
"int main(){int x;x=10;while(x>0)display(x);if(x>5)x=1;else x=0;return 0;}" | java -cp ".;antlr-4.13.2-complete.jar" Main4
```

Output

```
PS E:\Compiler design\Question 5> java -jar ".\antlr-4.13.2-complete.jar" MiniLang.g4
PS E:\Compiler design\Question 5> dir

Directory: E:\Compiler design\Question 5

Mode                LastWriteTime         Length Name
----                -
-a----             13-08-2026     11:26     2148045 antlr-4.13.2-complete.jar
-a----             13-08-2026      01:13        1504 Main4.java
-a----             13-08-2026      01:14         993 MiniLang.g4
-a----             13-08-2026      01:14        2422 MiniLang.interp
-a----             13-08-2026      01:14         271 MiniLang.tokens
-a----             13-08-2026      01:14        3044 MiniLangBaseListener.java
-a----             13-08-2026      01:14        3442 MiniLangLexer.interp
-a----             13-08-2026      01:14        8232 MiniLangLexer.java
-a----             13-08-2026      01:14         271 MiniLangLexer.tokens
-a----             13-08-2026      01:14        2218 MiniLangListener.java
-a----             13-08-2026      01:14        18970 MiniLangParser.java

PS E:\Compiler design\Question 5> javac -cp ".;antlr-4.13.2-complete.jar" *.java
PS E:\Compiler design\Question 5> "int main(){int x;x=10;while(x>0)display(x);if(x>5)x=1;else x=0;return 0;}" | java -cp ".;antlr-4.13.2-complete.jar" Main4
Accepted
```

Result

Thus, the parser successfully recognizes the complete mini programming language containing declarations, assignments, conditional statements, loops, function calls, and return statements, and displays Accepted or Rejected accordingly.